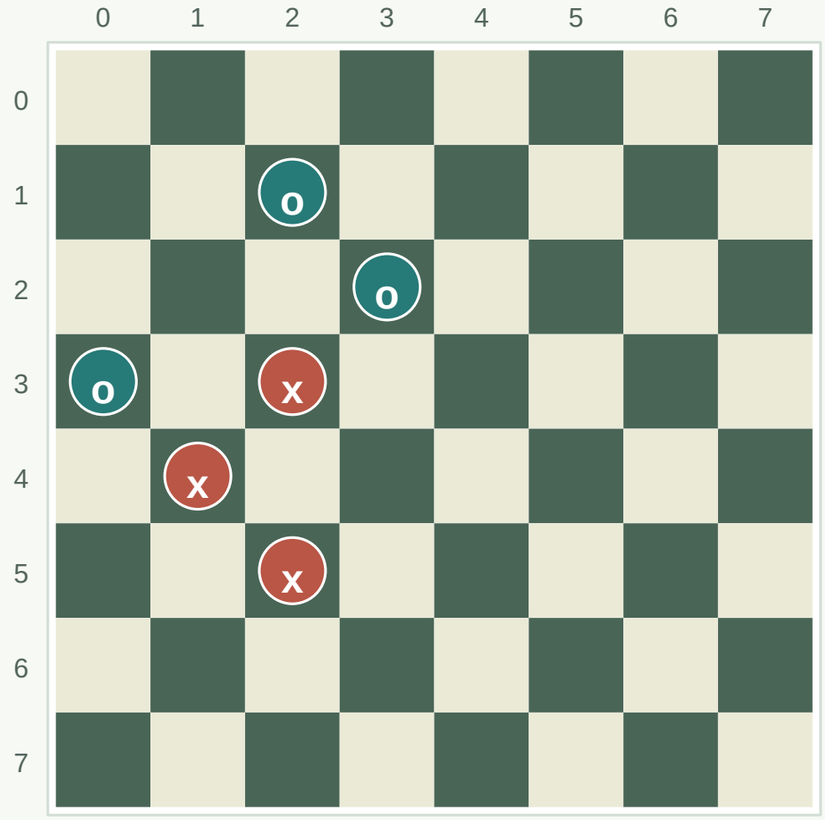
Watch the BOT think — on a real board

Starting position · BOT to move



o BOT · MAX

x HUMAN · MIN

Which move should O choose?

A (1,2) → (2,1)

One legal starting move

B (2,3) → (3,4)

The other legal starting move

We will follow the actual recursive calls, then see exactly where work is skipped.

Depth 3 means three simulated moves

1 · BOT candidate

findBestMove(depth 3)

Apply O’s move to a copy.

2 · Human response

minimax(depth 2)

Current player x → MIN.

3 · BOT reply

minimax(depth 1)

Current player o → MAX.

After the third move: minimax(depth 0) counts the pieces and returns a score.

```
Game copy = new Game(game);
copy.move(from, to);
minimax(copy, 2, 'o',
        Integer.MIN_VALUE, Integer.MAX_VALUE);
```

The first limits

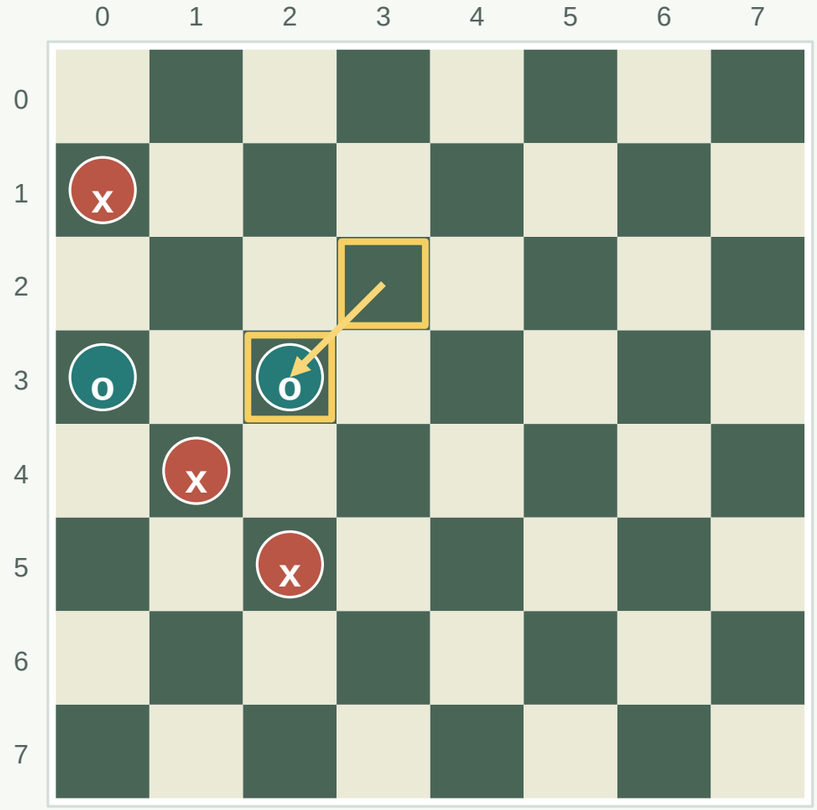
alpha = −2,147,483,648 (−∞)

beta = +2,147,483,647 (+∞)

The BOT calculates both sides. Game.move() decides whose simulated turn comes next.

Candidate A: follow one path to a real score

Leaf A1a · depth 0 · return −3



The moves on this simulated path

BOT	(1,2) → (2,1)
HUMAN	(3,2) → (1,0)
BOT	(2,3) → (3,2)

The human’s jump removed O at (2,1).

2 O × 3 − 3 X × 3 = −3

The leaf returns −3 to its waiting caller. No real game piece has moved.

Candidate A finishes and returns −3

Human response A1

(3,2) → (1,0)

BOT replies: −3, −3

MAX returns −3

Human response A2

(3,2) → (1,4)

BOT reply: −3

MAX returns −3

The human MIN call returns MIN(−3, −3) = −3.

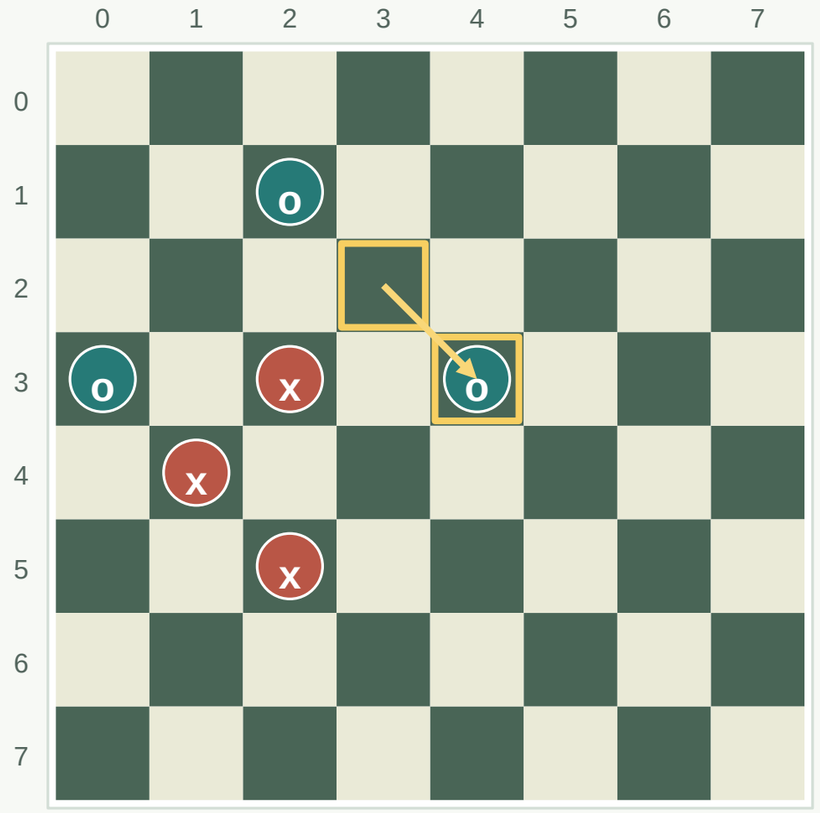
```
// Back inside findBestMove()
score = -3;
bestScore = -3;
bestMove = A;
```

A is the best initial move found so far.

minimax() returns a score. findBestMove() remembers which initial move produced it.

Candidate B starts with fresh limits

After B · human turn · depth 2



HUMAN MIN · depth 2 · ACTIVE

`alpha = −∞` `beta = +∞`

`minEval = +∞`

Three legal human responses

H1 (3,2) → (2,1)

H2 (3,2) → (2,3)

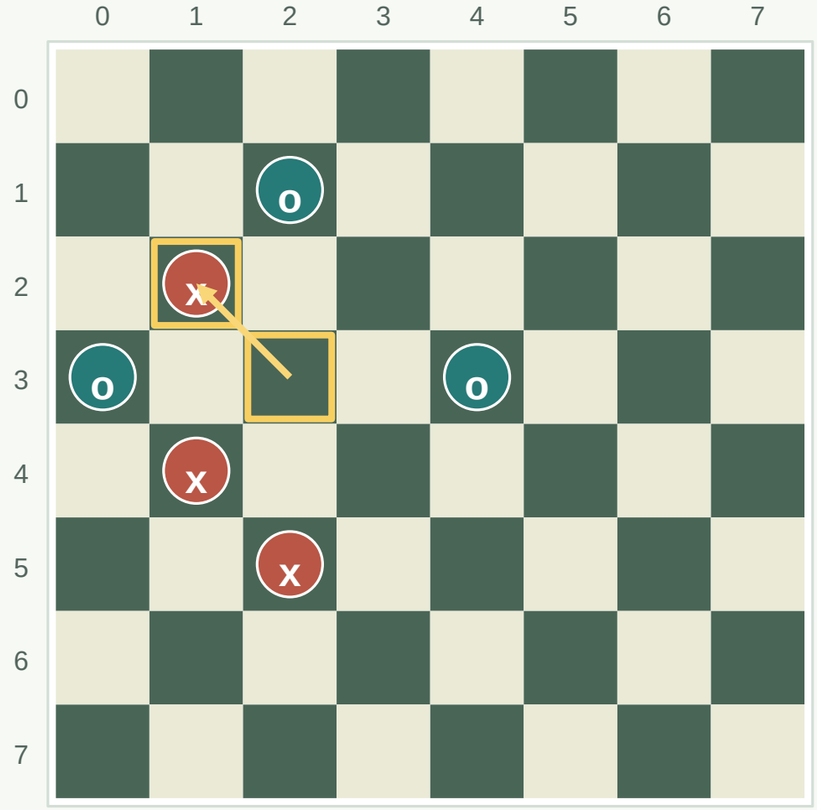
H3 (5,2) → (4,3)

Your code resets alpha and beta for B; A's score stays only in bestScore.

All search moves are simulated. Every score is from the BOT's perspective.

H1 creates a child call; its parent waits

H1: X (3,2) → (2,1)



HUMAN MIN · depth 2 · PAUSED

alpha = −∞ beta = +∞

`minEval = +∞`

CALL ↓ minimax(copy, 1, 'o', −∞, +∞)

BOT MAX · depth 1 · ACTIVE

alpha = −∞ beta = +∞

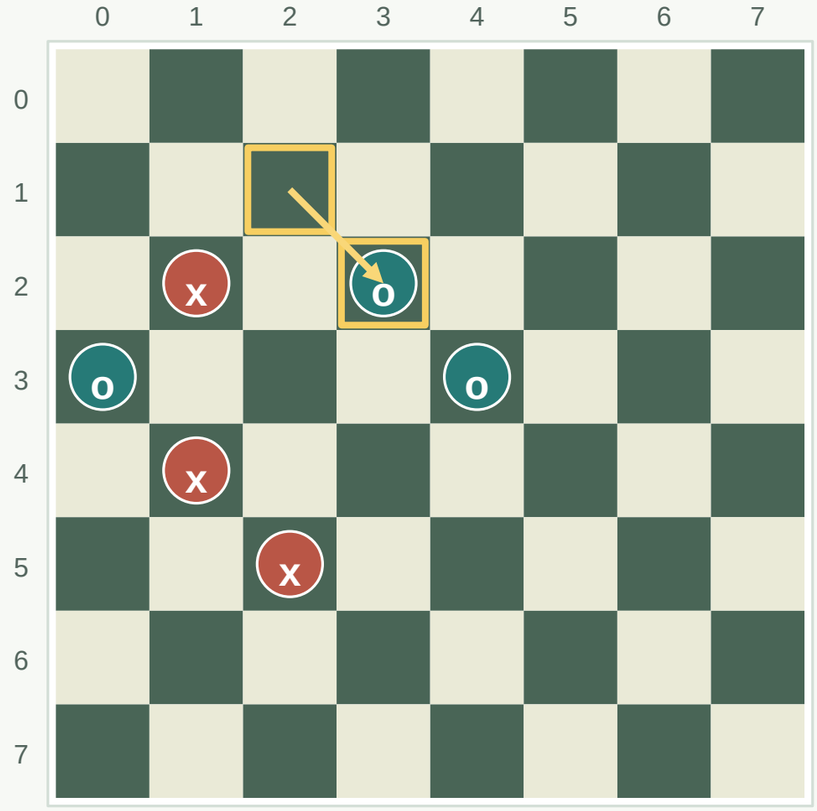
`maxEval = −∞`

```
int eval = minimax(...);
// The parent pauses on this line.
```

A new call owns new variables. The caller resumes only after that call returns.

The first H1 leaf returns 0

BOT reply: O (1,2) → (2,3)



LEAF · depth 0 · no winner

3 O × 3 − 3 X × 3 = 0

```
if (depth <= 0) {
    return evaluateBoard(game, botSymbol);
}
```

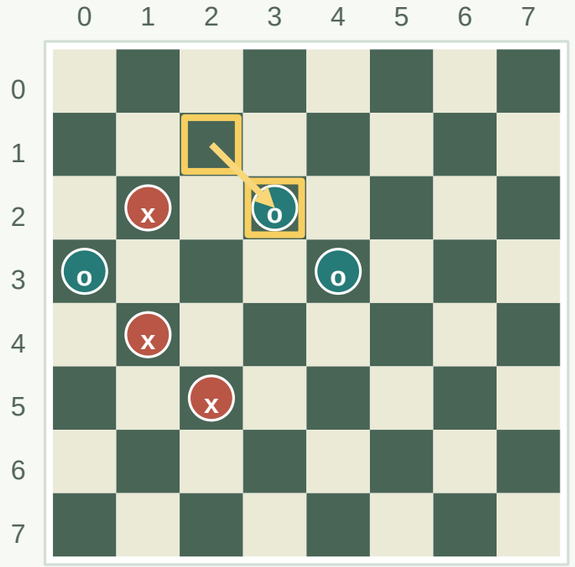
RETURN 0 → resume the waiting BOT MAX call

```
eval = 0; // returned child score
maxEval = Math.max(maxEval, eval); // 0
alpha = Math.max(alpha, eval);     // 0
```

The score 0 is fully calculated for this reply, up to the chosen depth.

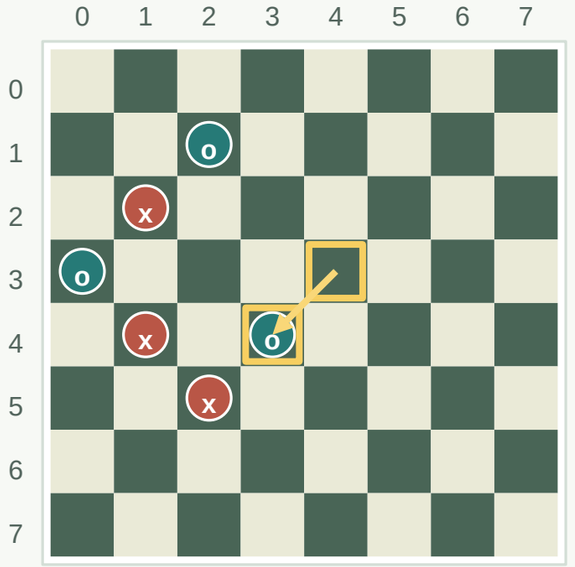
Finish H1, then give its result to the human MIN call

Reply 1 · score 0



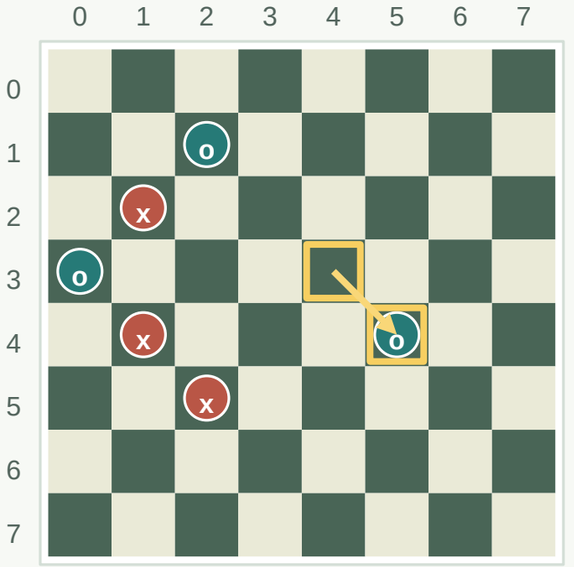
(1,2) → (2,3)

Reply 2 · score 0



(3,4) → (4,3)

Reply 3 · score 0



(3,4) → (4,5)

Alternatives

Each reply starts from the same H1 board.

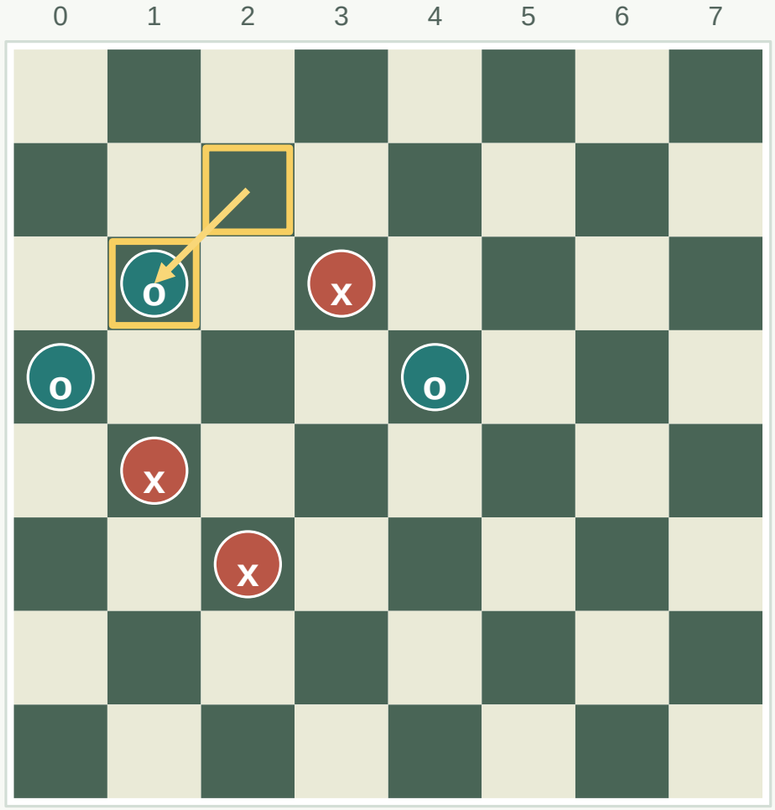
MAX(0,0,0) = 0

H1 returns 0 → human MIN resumes: `minEval = 0` `beta = 0`

beta = 0 means the simulated human already has H1, which leaves the BOT with 0.

H2: even equality can skip work

O (1,2) → (2,1) · returns 0



H2 is X (3,2) → (2,3).

BOT MAX · depth 1 · ACTIVE

alpha = 0 beta = 0

maxEval = 0

```
if (beta <= alpha) {
    break;  // 0 <= 0 is true
}
```

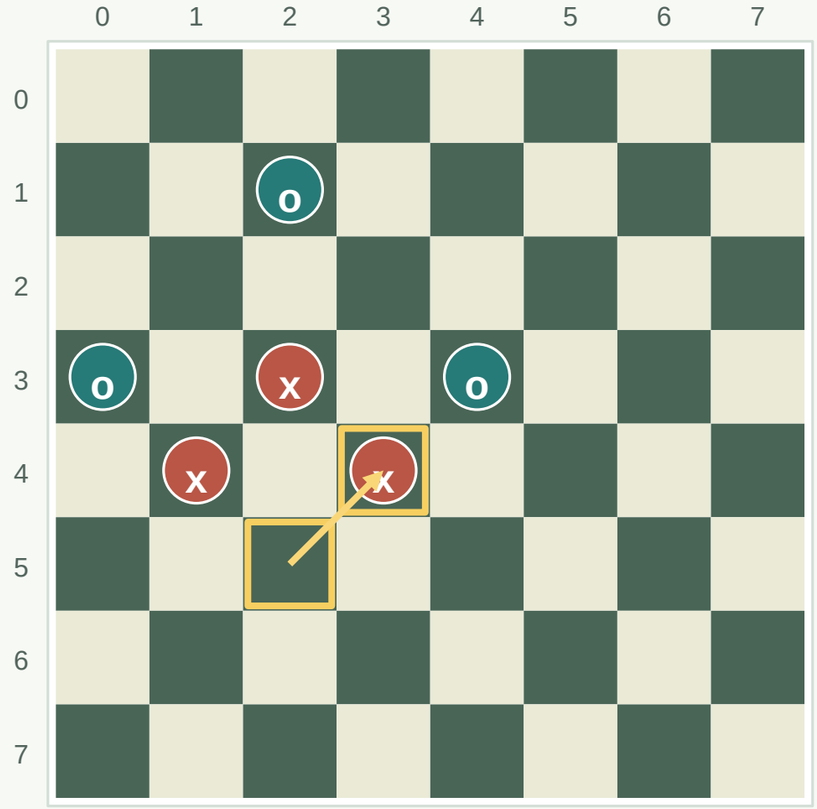
Skip two remaining replies under H2.

The human can already keep score 0 through H1.

H2's final MAX is at least 0. It cannot force a score below the human's existing 0.

H3 gives the BOT two legal captures

H3: X (5,2) → (4,3)



BOT MAX · depth 1 · ACTIVE

alpha = −∞ beta = 0

maxEval = −∞

First BOT capture · B3a

(3,0) → (5,2)

Remove X at (4,1).

Second BOT capture · B3b

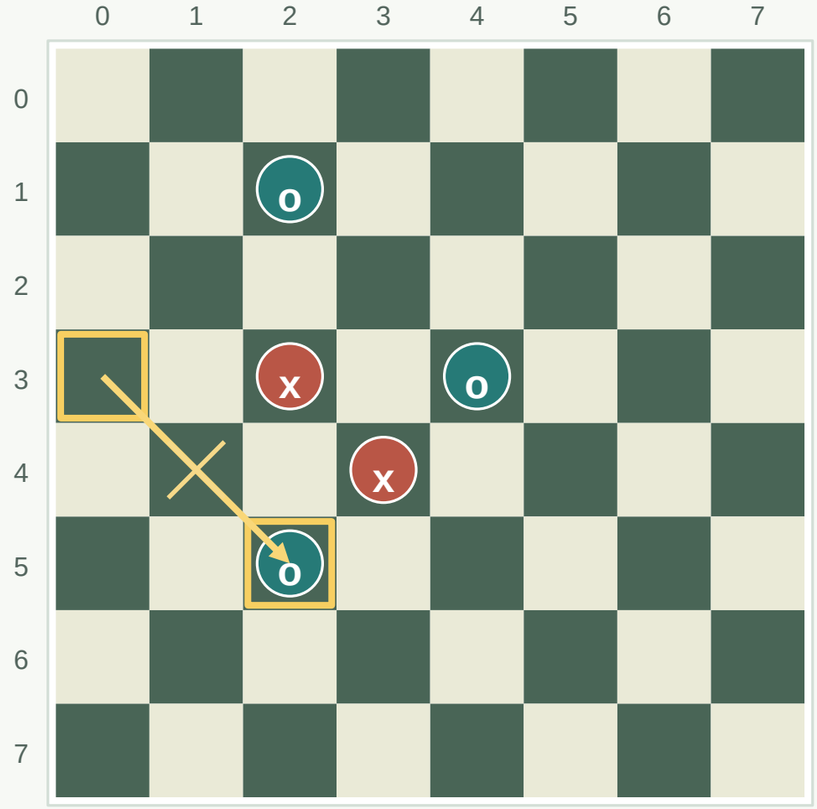
(3,4) → (5,2)

Remove X at (4,3).

These are two alternative replies from H3. Each would use its own board copy.

The first capture finishes and returns +3

B3a · O (3,0) → (5,2)



One X piece was captured.

3 O × 3 − 2 X × 3 = +3

The depth-zero call returns 3.

```
eval = 3; // returned child score
maxEval = Math.max(maxEval, eval); // 3
alpha = Math.max(alpha, eval);     // 3
```

H3 now has alpha = 3, beta = 0.

The second capture has not been evaluated.

The recursive call for B3a has finished. The larger H3 subtree is still unfinished.

Why another reply cannot lower the MAX result

H1 · already finished

The BOT’s best reply gives 0.

The human can choose this option.

H3 · partly explored

The BOT already has a reply worth 3.

Its final MAX must be at least 3.

```
maxEval = Math.max(3, anotherReplyScore);
```

A lower alternative leaves MAX at 3. A higher alternative raises it.

The BOT predicts the human will keep 0 through H1, so the rest of H3 is unnecessary.

The condition ends this loop, then returns to its caller

```
int eval = minimax(...); // returns 3
maxEval = Math.max(maxEval, eval);
alpha = Math.max(alpha, eval);
if (beta <= alpha) {      // 0 <= 3
    break;               // skip B3b
}
return maxEval;           // return 3
```

BOT MAX · depth 1 · RETURNING

alpha = 3 **beta = 0**

maxEval = 3

RETURN 3 ↓

HUMAN MIN · depth 2 · RESUMES

alpha = −∞ **beta = 0**

minEval = 0

The human caller resumes:

minEval = MIN(0, 3) = 0

break skips one loop’s remaining replies. The other waiting calls continue.

B returns 0, so the BOT chooses B

Human MIN inside B

H1 → 0

H2 → at least 0

H3 → at least 3

B’s score = 0

findBestMove()

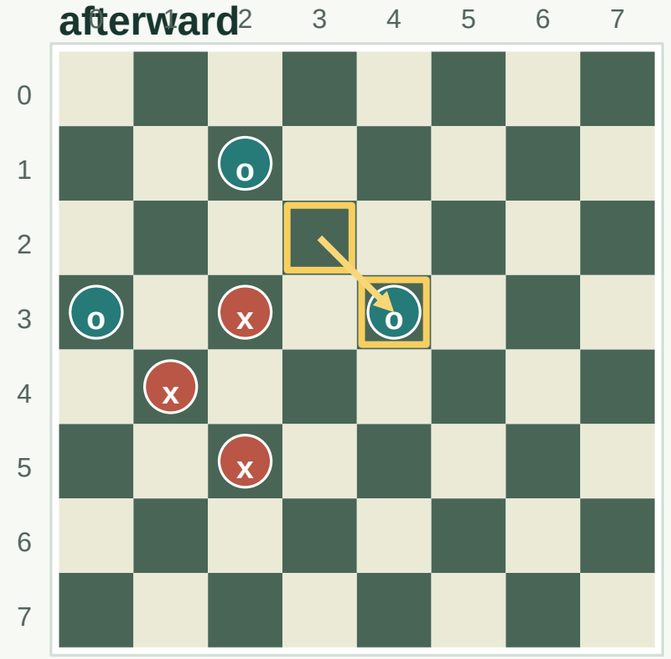
A returned −3

B returned 0

0 > −3

bestMove = B

Chosen move applied afterward

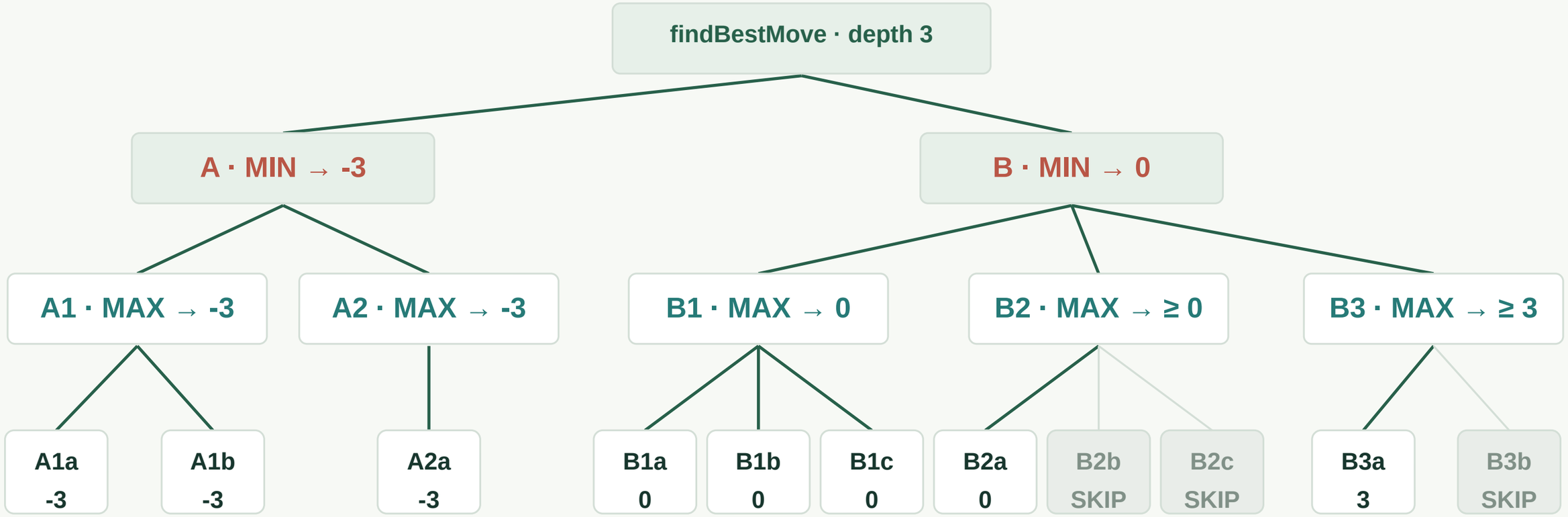


O (2,3) → (3,4)

```
return bestMove;  // B
```

The search returns a move. DamkaFX then calls applyLocalMove() to play it.

The complete tree — and the work actually saved



Without pruning: 18 minimax calls · 11 leaf evaluations

With alpha–beta: 15 calls · 8 leaf evaluations · the same chosen move B.

All search moves are simulated. Every score is from the BOT's perspective.

Now take control of the calls yourself

1 · Open the HTML file

Use Next / Previous, or the arrow keys.

“Jump to strict cutoff” replays H3.

2 · Before every Next

Predict: which call is active?

What returns? Which variable changes?

3 · Watch the call stack

A parent waits while its child runs.

Each box owns its depth, alpha and beta.

4 · Read your Java code

BotPlayer.findBestMove() → minimax()

Game.move() → currentPlayer

At H3: alpha = ? beta = ? why can B3b be skipped?

Answer: alpha 3, beta 0. H3’s MAX cannot fall below 3; H1 already gives the human 0.